

Pipeline de Observabilidad End-to-End con OpenTelemetry

Entregable: Reporte Técnico · Cloud: GCP Cloud Run + Cloud SQL PostgreSQL · Fecha: Agosto 2026

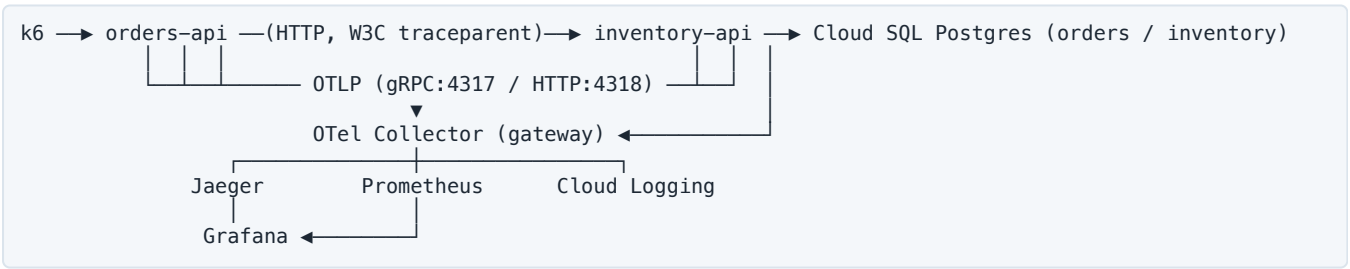
1. Resumen ejecutivo

Se implementó y validó un pipeline de observabilidad end-to-end basado en OpenTelemetry sobre dos microservicios Python/FastAPI (`orders-api` → `inventory-api`), emitiendo las tres señales —trazas, métricas y logs estructurados— hacia un OpenTelemetry Collector que las enruta a Jaeger, Prometheus y stdout en local, y a Cloud Trace, Managed Prometheus y Cloud Logging en la nube, con visualización unificada en Grafana. El requisito crítico de propagación W3C Trace Context se verificó empíricamente: un mismo `trace_id` correlaciona logs, trazas y métricas a través de ambos servicios, tanto en local como en la nube.

El despliegue se realizó en **GCP Cloud Run + Cloud SQL PostgreSQL**, con CI/CD por GitHub Actions autenticado mediante Workload Identity Federation (sin llaves de servicio) y secretos en Secret Manager, bajo una estrategia de costo que apaga Cloud SQL fuera de sesión para respetar los créditos de la Free Trial. El benchmark de overhead (k6) demostró un costo de instrumentación medible: -38% de throughput, +31% de latencia p99 y +19-27% de memoria, mitigable con muestreo de trazas y batching. Las pruebas de carga en la nube demostraron además el valor diagnóstico del pipeline al aislar y corregir un agotamiento de conexiones en Cloud SQL.

2. Arquitectura

La solución se compone de dos microservicios con dependencia HTTP, cada uno con acceso a su propia base de datos, y un plano de observabilidad desacoplado vía OTLP.



Componente	Rol	Puerto
<code>orders-api</code> (service-a)	Crea pedidos, llama a inventory por HTTP, persiste la orden	8080
<code>inventory-api</code> (service-b)	Valida y reserva stock (UPDATE atómico)	8081
OTel Collector	Recibe OTLP y enruta las 3 señales (batch + memory_limiter + resource)	4317/4318/8889
Jaeger all-in-one	Almacenamiento y visualización de trazas (local)	16686
Prometheus	Métricas (scrape del collector :8889 y :8888)	9090
Grafana	Dashboard SLI/SLO + correlación por <code>trace_id</code>	3000
Cloud Logging	Logs JSON en la nube, consultables por <code>trace_id</code>	—

Cloud SQL PostgreSQL	Bases <code>orders</code> e <code>inventory</code> (una por servicio)	5432
----------------------	---	------

Flujo de una orden: el cliente hace `POST /orders` → `orders-api` valida la entrada (Pydantic) → llama por HTTP a `inventory-api` (`POST /products/{id}/reserve`) propagando el `traceparent` → `inventory-api` descuenta stock con un `UPDATE` atómico → `orders-api` inserta la orden y responde 201. Cada salto genera spans de auto-instrumentación más spans custom de negocio.

3. Instrumentación con OpenTelemetry

3.1 Auto-instrumentación

Ambos servicios se ejecutan bajo `opentelemetry-instrument` (CLI de `opentelemetry-distro`), que aplica instrumentación automática de FastAPI (spans server), httpx (spans client, con propagación W3C automática) y SQLAlchemy (spans connect/execute con atributos `db.statement`, `db.system`). Versiones pineadas: SDK 1.29.0, instrumentaciones 0.50b0.

3.2 Spans custom y métrica custom

Señal	Elemento	Servicio	Propósito
Trazas	<code>persist_order</code>	orders-api	Span de negocio sobre el INSERT de la orden
Trazas	<code>check_stock</code>	inventory-api	Span sobre la consulta de stock
Trazas	<code>reserve_stock</code>	inventory-api	Span sobre el descuento atómico de stock
Métricas	<code>db_operation_duration</code>	ambos	Histograma (ms) de latencia DB — necesario porque SQLAlchemy async no emite <code>db.client.operation.duration</code>

3.3 Logs estructurados

Logs en JSON (structlog) con un processor que inyecta `trace_id` y `span_id` desde el span activo:

```
{"order_id": "da65151f-...", "event": "order.created", "trace_id": "3d6cf90ef96226881054ba92f6594942", "span_id": "be787879b17fac07", "level": "info", "timestamp": "2026-08-17T02:07:05.350200Z"}
```

4. Verificación de la correlación cross-signal (W3C Trace Context)

El requisito crítico es que el `trace_id` sea consistente entre las tres señales. Se validó con un trace real (`3d6cf90ef96226881054ba92f6594942`):

- **Logs:** el JSON de `order.created` contiene el mismo `trace_id` (arriba).
- **Trazas:** Jaeger muestra el árbol completo con 15 spans; los spans de inventory-api son hijos del span `POST /orders` de orders-api, confirmando la propagación `traceparent` vía httpx.
- **Métricas:** las métricas se etiquetan por `service_name` gracias a `resource_to_telemetry_conversion: enabled` en el exporter Prometheus.

La misma verificación se repitió en la nube: una traza distribuida en Cloud Trace abarca spans de ambos servicios Cloud Run bajo un único `trace_id`, y Cloud Logging permite filtrar los logs por ese mismo identificador.

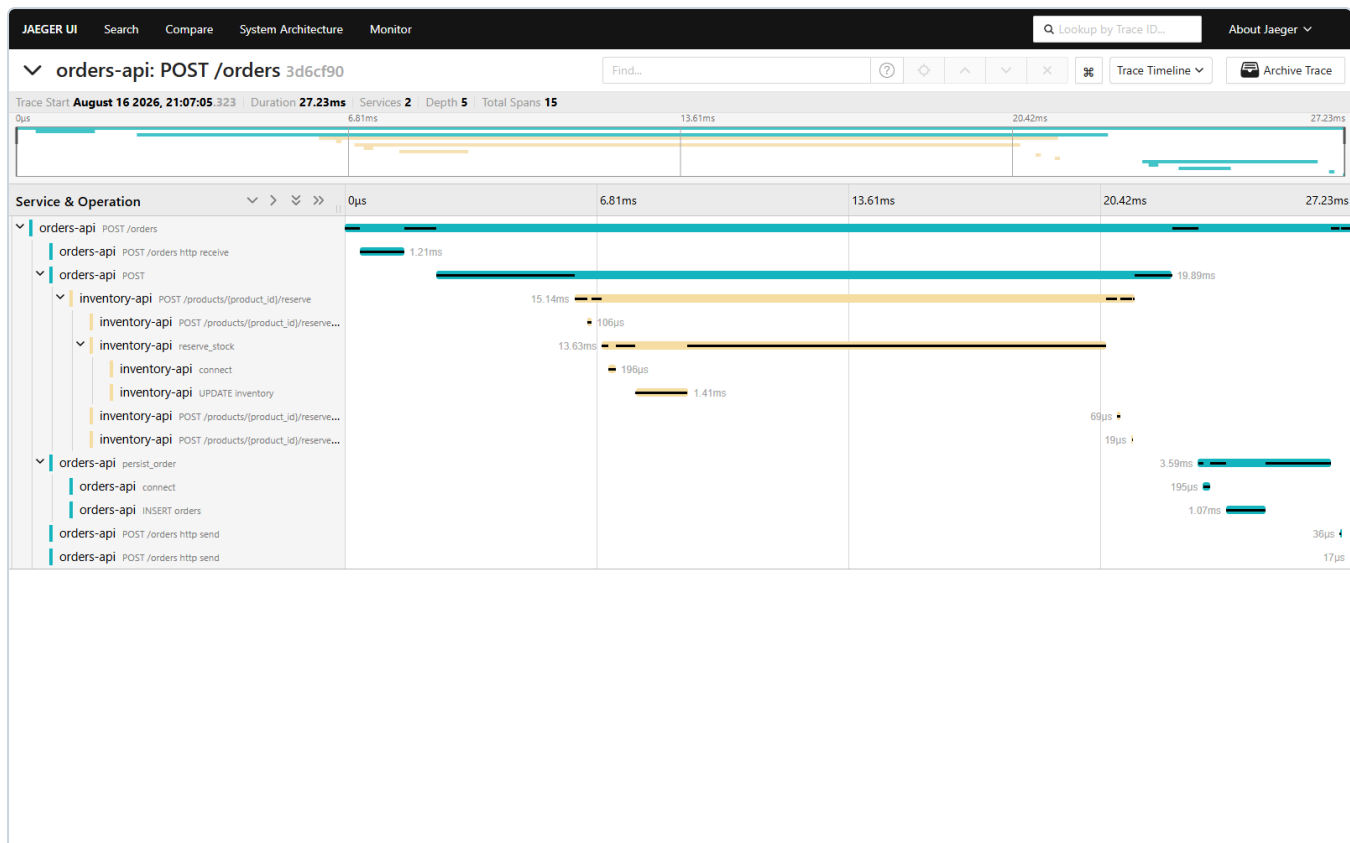


Figura 1 — Traza distribuida en Jaeger: spans de orders-api (FastAPI, httpx, SQLAlchemy, persist_order) y de inventory-api (reserve_stock), unificadas por un único trace_id (propagación W3C).

5. Backends y visualización

5.1 OTEL Collector

Configuración con receiver OTLP (gRPC + HTTP), processors `batch` + `memory_limiter` + `resource` y exporters `otlp/jaeger` (trazas), `prometheus` (:8889) y `debug` (logs en local). En GCP, un segundo Collector desplegado como servicio de Cloud Run recibe OTLP (gRPC sobre TLS) y exporta con `googlecloud` (trazas a Cloud Trace, logs a Cloud Logging) y `googlemanagedprometheus` (métricas a Managed Prometheus). Nota: el exporter `jaeger` fue eliminado en el collector-contrib ≥ 0.109 , por lo que se usa `otlp` hacia el receptor OTLP de Jaeger (`COLLECTOR_OTLP_ENABLED=true`).

5.2 Dashboard Grafana (6 paneles)

Panel	Señal	Consulta Prometheus
Latencia p99 HTTP	SLI	<code>histogram_quantile(0.99, sum by (le, service_name) (rate(http_server_duration_milliseconds_bucket[1m])))</code>
Error rate (5xx)	SLI	<code>sum(rate(..._count{http_status_code=~"5.."}[1m])) / sum(rate(..._count[1m]))</code>
Throughput (RPS)	SLI	<code>sum by (service_name) (rate(http_server_duration_milliseconds_count[1m]))</code>
Latencia DB p99	SLI	<code>histogram_quantile(0.99, sum by (le, service_name) (rate(db_operation_duration_milliseconds_bucket[1m])))</code>
CPU OTEL Collector	Infra	<code>rate(otelcol_process_cpu_seconds[1m])</code>
Errores del Collector	Infra	<code>sum by (exporter) (rate(otelcol_exporter_send_failed_spans[1m]))</code>

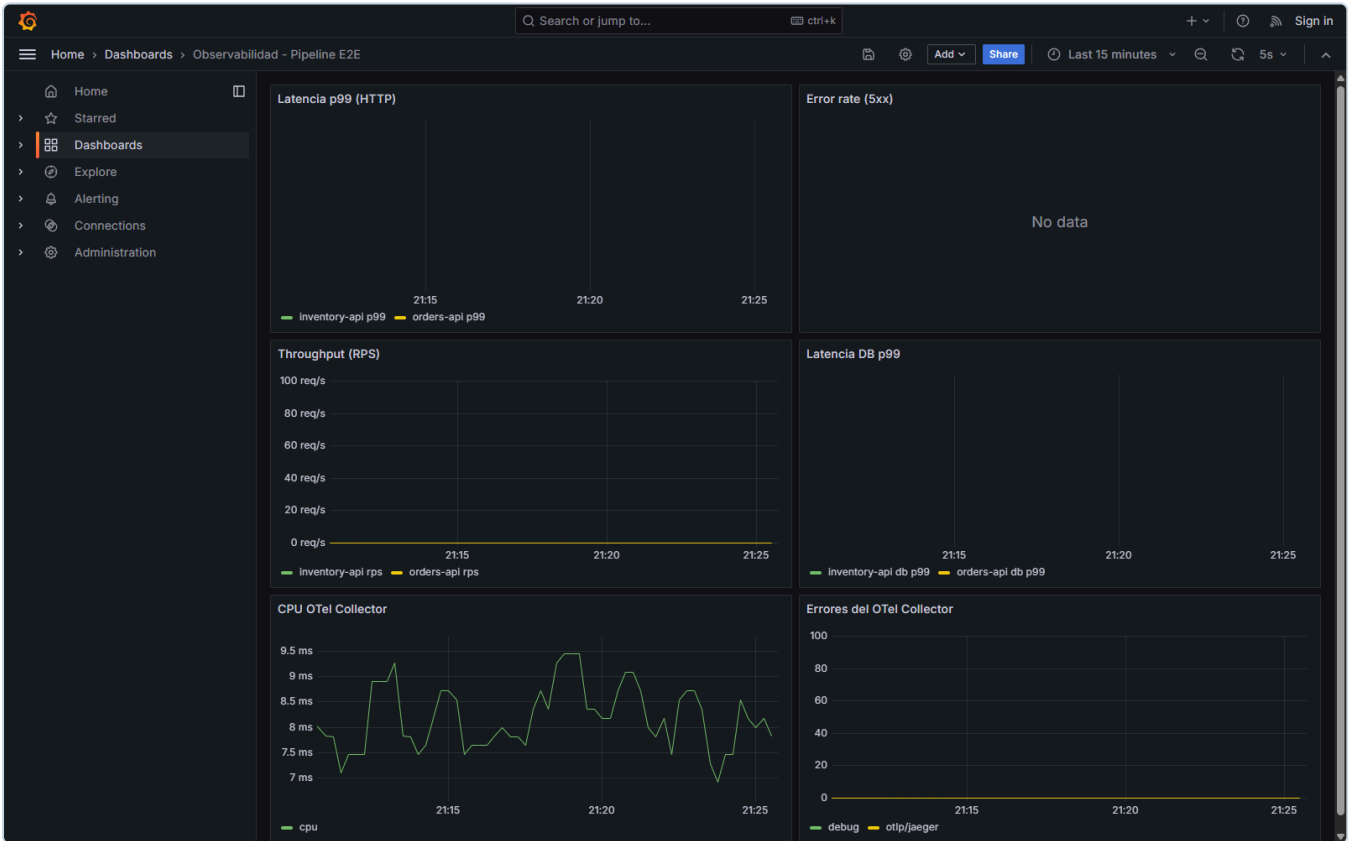


Figura 2 — Dashboard Grafana provisionado: 4 SLIs + CPU del collector + errores del collector, alimentado por Prometheus con datasource Jaeger para correlación.

6. Análisis de overhead (Fase 4 — k6)

6.1 Metodología

- **Con instrumentación:** servicios bajo `opentelemetry-instrument` (auto-instrumentación + spans custom + métricas OTLP cada 5s).
- **Sin instrumentación:** los mismos servicios con `uvicorn` directo (override de `compose`); el código de telemetría queda en no-op.
- **Carga:** k6, 50 VUs constantes sobre `POST /orders`, `product_id` rotado entre 100 productos para evitar contención de fila.

6.2 Resultados (local, Docker Desktop Windows)

Métrica	Sin OTel	Con OTel	Overhead
Throughput	217 req/s	135 req/s	-38%
Latencia promedio	229 ms	368 ms	+61%
Latencia p95	525 ms	900 ms	+71%
Latencia p99	945 ms	1240 ms	+31%
Memoria orders-api	72 MiB	86 MiB	+19%
Memoria inventory-api	64 MiB	81 MiB	+27%
CPU (1 worker)	~97%	~98%	similar (ver 6.3)

6.3 Interpretación

El CPU se mantiene saturado en ambos casos (un único worker de `uvicorn`); el costo real de CPU se refleja en la caída de throughput a CPU comparable: más CPU por request.

- **Span fan-out alto:** cada `POST /orders` genera ~15–16 spans (~2000 spans/s a 135 req/s) exportados por OTLP.
- El export de métricas cada 5s (`OTEL_METRIC_EXPORT_INTERVAL=5000`) suma overhead de serialización bajo carga.
- **Sin muestreo** (default siempre-muestrea): con tail-based sampling (10–25%) el costo de export de trazas cae drásticamente.

7. Despliegue en GCP (Cloud Run + Cloud SQL)

El despliegue se realizó y validó sobre **Google Cloud Platform** (proyecto dedicado con créditos de la Free Trial), sustituyendo el plan original en AWS por una arquitectura serverless equivalente y más simple de operar:

- **Cloud Run** para los cuatro servicios: `orders-api`, `inventory-api`, el **OTel Collector** (hub de observabilidad, gRPC sobre TLS/HTTP2) y **Grafana**. Todos escalan a cero: sin tráfico, costo ≈ \$0.
- **Cloud SQL PostgreSQL 16** (`db-f1-micro`, `single-AZ`), conexión por socket Unix (`--add-cloudsql-instances`), bases `orders` e `inventory`. Se apaga fuera de sesión (`activation-policy=NEVER`) como guardrail de coste.
- **CI/CD con GitHub Actions**: el merge a `main` despliega automáticamente el servicio cuyo código cambió (filtro de rutas), con alternativas de despliegue manual (`workflow_dispatch`) y por script local sin subir código. La autenticación GitHub→GCP usa **Workload Identity Federation**: sin llaves de servicio almacenadas, con acceso restringido por condición al repositorio.
- **Secret Manager** para las cadenas de conexión (`DATABASE_URL`): Cloud Run las inyecta en runtime; nunca pasan por GitHub.
- **Telemetría nativa**: el Collector exporta trazas a **Cloud Trace**, logs a **Cloud Logging** (consultables por `trace_id`) y métricas a **Managed Prometheus**, que Grafana consulta con PromQL mediante el datasource de Google Cloud Monitoring.

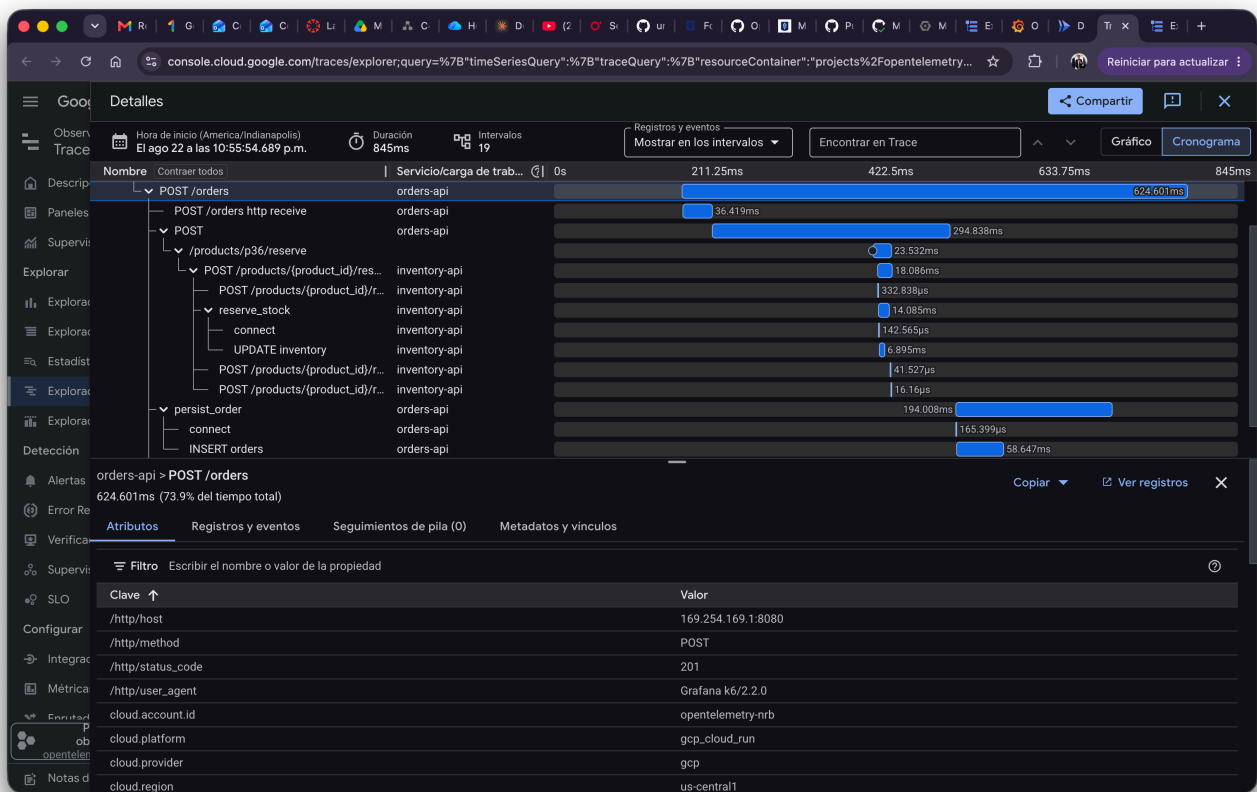


Figura 3 — Traza distribuida en Cloud Trace (845 ms, HTTP 201): el árbol abarca `orders-api` e `inventory-api` bajo un mismo `trace_id`, con los spans custom (`reserve_stock`, `persist_order`), las consultas SQL (`UPDATE inventory`, `INSERT orders`) y atributos de recurso `gcp_cloud_run` — la propagación W3C funcionando en la nube.

7.1 Pruebas de carga en la nube y diagnóstico con el propio pipeline

Se ejecutaron pruebas k6 de 2 minutos sostenidos contra el despliegue real, que produjeron un incidente diagnosticado íntegramente con la telemetría del pipeline:

Prueba	Éxito	RPS	p95	Hallazgo
50 VUs (pools 20/40)	14.4%	9	10 s	TooManyConnectionsError: los pools dimensionados para local agotaron las ~25 conexiones de db-f1-micro
15 VUs (pools 5/5)	100%	74	326 ms	Fix validado: pool configurable por entorno (DB_POOL_SIZE / DB_MAX_OVERFLOW), cero errores
80 VUs (pools 5/5)	39%	142	1.38 s	El autoescalador multiplicó instancias (hasta 9) y con ellas las conexiones; instancias nuevas fallaron el arranque (conexión a DB en el startup)

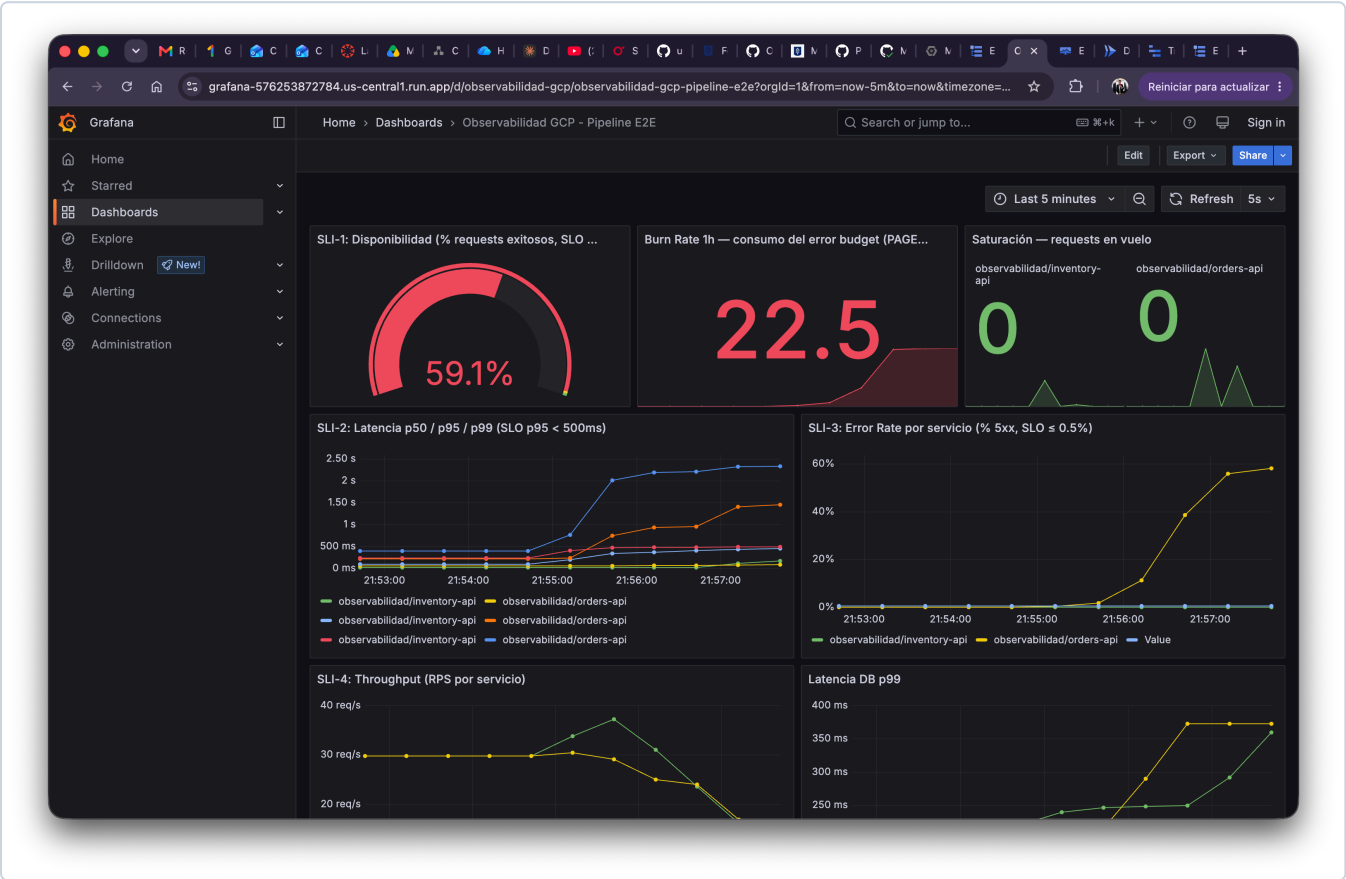


Figura 4 — Dashboard Grafana (Cloud Run) durante la prueba de 80 VUs: disponibilidad cae a 59.1% (gauge en rojo), burn rate 22.5 (> 14.4 = alerta PAGE), latencias p50/p95/p99 disparadas por encima del SLO y error rate creciendo — los 4 SLIs capturando el incidente en tiempo real.

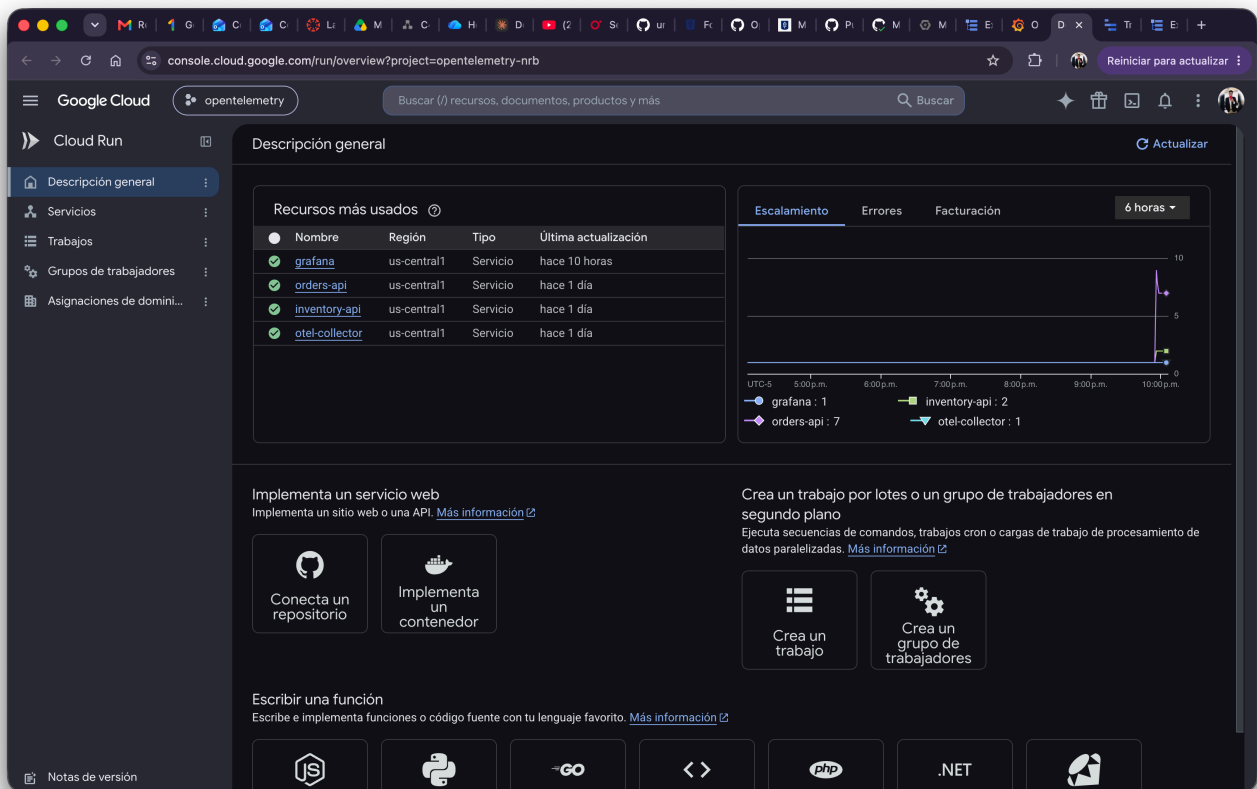


Figura 5 — Consola de Cloud Run: los cuatro servicios del pipeline (orders-api, inventory-api, otel-collector, grafana) y el gráfico de escalamiento donde orders-api salta de 1 a 7+ instancias durante la carga — el multiplicador de conexiones que agotó Cloud SQL.

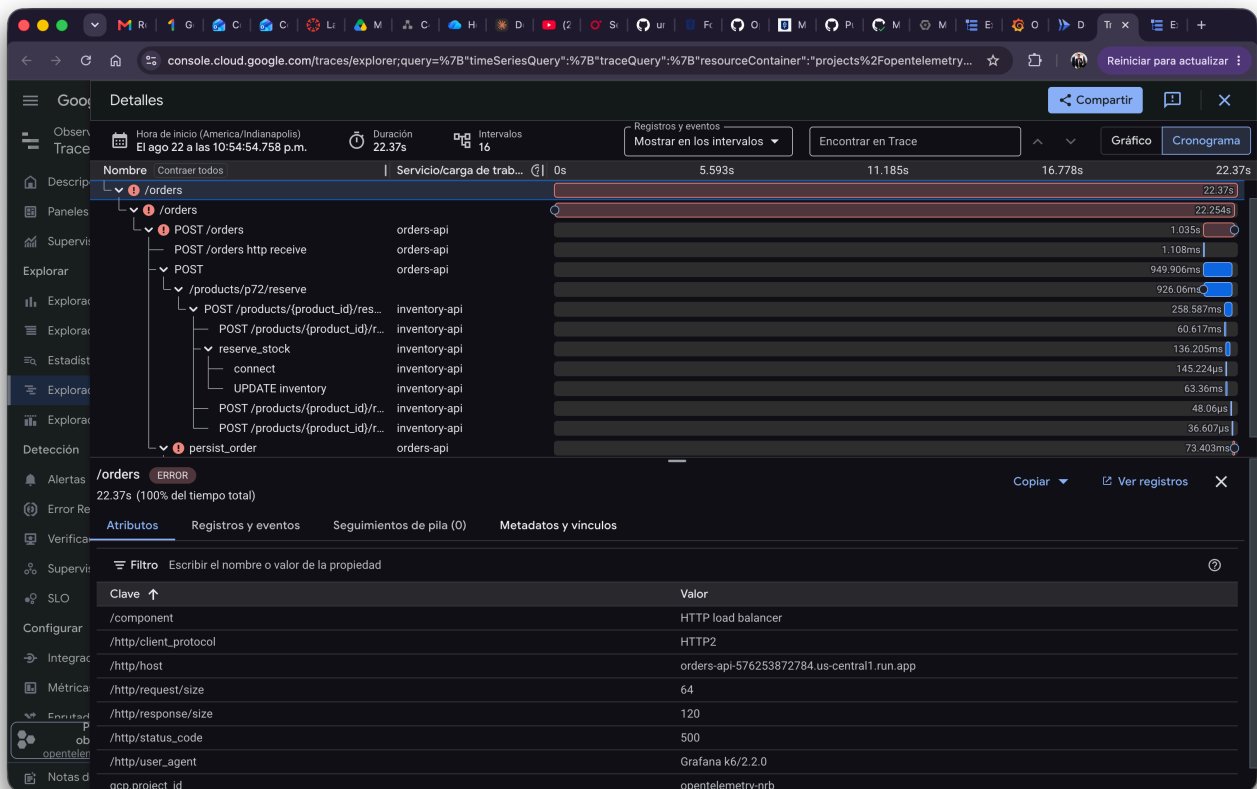


Figura 6 — Traza del incidente en Cloud Trace: un POST /orders de 22.37 s terminando en HTTP 500; la cascada muestra la espera masiva antes del trabajo real (el pool de conexiones agotado), evidencia con la que se diagnosticó la causa raíz.

Lecciones de la nube: el pool de conexiones debe dimensionarse por instancia contando el autoescalado (o limitarse `max-instances`), y el arranque que se conecta a la base de datos es frágil bajo presión — la conexión perezosa con reintentos es la mitigación recomendada.

8. Decisiones de diseño y lecciones aprendidas

Problema encontrado	Solución aplicada
Exporter <code>jaeger</code> eliminado en collector-contrib 0.109	Exporter <code>otlp</code> → receptor OTLP de Jaeger
Imagen del collector distroless (sin shell/wget)	Sin healthcheck; el exporter OTLP reintenta solo
Métricas sin label <code>service_name</code>	<code>resource_to_telemetry_conversion: enabled</code>
SQLAlchemy async no emite <code>db.client.operation.duration</code>	Histograma custom <code>db_operation_duration</code>
Métricas con lag de 60s	<code>OTEL_METRIC_EXPORT_INTERVAL=5000</code>
Contención de fila en el benchmark (todas las VUs sobre p1)	Repartir carga entre 100 productos — detectado vía trazas
Latencia HTTP elevada por conexión nueva por request	<code>httpx.AsyncClient</code> compartido (reutilización de conexiones)
Pools de DB dimensionados para local agotaban Cloud SQL (~25 conexiones en <code>db-f1-micro</code>)	Pool configurable por entorno: <code>DB_POOL_SIZE / DB_MAX_OVERFLOW = 5/5</code> en Cloud Run
Credenciales de despliegue en CI	Workload Identity Federation (sin llaves) + Secret Manager para la DB
Cloud Run servía imagen cacheada bajo el tag <code>:latest</code>	Despliegue pineado por digest de imagen

9. Conclusiones

Se logró un pipeline de observabilidad completo y funcional con OpenTelemetry sobre microservicios FastAPI, cubriendo las tres señales y la correlación por `trace_id` exigida por el estándar W3C Trace Context. Las trazas distribuidas no solo cumplen el requisito de propagación, sino que demostraron su valor operativo al permitir aislar cuellos de botella (contención de fila y sobrecarga de conexiones) durante el benchmark. El overhead de instrumentación es medible y aceptable a baja carga; bajo carga alta, el span fan-out y la ausencia de muestreo lo amplifican, siendo el tail-based sampling la mitigación recomendada para producción. El despliegue en **GCP Cloud Run + Cloud SQL**, con CI/CD sin llaves (Workload Identity Federation), secretos en Secret Manager y apagado de la base de datos fuera de sesión, mantiene el costo dentro de los créditos de la Free Trial; las pruebas de carga en la nube validaron además el valor diagnóstico del pipeline al aislar un agotamiento de conexiones en Cloud SQL y guiar su corrección hasta una prueba con 100% de éxito.